

ASSIGNMENT-1.5

Name: Kaveti Sreenavya

HT.No: 2303A51244

Batch: 19

Task 1: AI-Generated Logic Without Modularization (String Reversal Without Functions)

Prompt: #String Reversal Without using functions

```
# String Reversal Without using functions
string = "Hello, World!"
reversed_string = ""

for char in string:
    reversed_string = char + reversed_string

print(reversed_string)

# Output: !dlroW ,olleH
```

OUTPUT:

```
PS C:\Users\v1\OneDrive\Desktop\AI Coding-2026> & "C:/Program Files/Python311/python.exe" "c:/Users/v1/OneDrive/Desktop/AI Coding-2026/day-02.py"
!dlroW ,olleH
PS C:\Users\v1\OneDrive\Desktop\AI Coding-2026> █
```

Observation: AI-Generated Logic Without Modularization

The Copilot-generated string reversal program correctly takes user input and reverses it without using functions. It works well for all types of input and provides clear, readable code suggestions. This shows how simple text-processing tasks can be easily implemented and how AI tools help beginners learn basic programming logic.

Task 2: Efficiency & Logic Optimization (Readability Improvement)

Prompt: #optimised the code

```
"""
#optimize this code
string = "Hello, World!"
reversed_string = ''.join(reversed(string))
print(reversed_string)
"""
```

OUTPUT:

```
PS C:\Users\v1\OneDrive\Desktop\AI Coding-2026> & "C:/Program Files/Python311/python.exe" "c:/Users/v1/OneDrive/Desktop/AI Coding-2026/day-02.py"
!dlroW ,olleH
!dlroW ,olleH
PS C:\Users\v1\OneDrive\Desktop\AI Coding-2026> █
```

Observation: Efficiency & Logic Optimization

The optimized string reversal code is easier to read and more efficient because it removes unnecessary variables and uses Python's built-in string slicing instead of a loop. This reduces complexity and makes the code simpler to understand and maintain. The task shows how Copilot prompts like "simplify logic" help generate cleaner and more efficient code.

Task 3: Modular Design Using AI Assistance (String Reversal Using Functions)

Prompt: String Reversal Using Functions

```
1 #String Reversal Using Functions
2 def reverse_string(s):
3     return s[::-1]
4 input_string = "Hello, World!"
5 reversed_string = reverse_string(input_string)
6 print("Original String:", input_string)
7 print("Reversed String:", reversed_string)
8 #Output:
9 #Original String: Hello, World!
10 #Reversed String: !dlroW ,olleH
```

OUTPUT:

```
PS C:\Users\v1\OneDrive\Desktop\AI Coding-2026> & "C:/Program Files/Python311/python.exe" "c:/Users/v1/OneDrive/Desktop/AI Coding-2026/day-02.py"
!dlroW ,olleH
!dlroW ,olleH
PS C:\Users\v1\OneDrive\Desktop\AI Coding-2026> █
```

Observation: Modular Design Using AI Assistance

The function-based string reversal program generated with GitHub Copilot demonstrates improved modularity and reusability compared to the earlier inline implementations. By encapsulating the reversal logic inside a user-defined function, the code becomes easier to maintain, test, and reuse across multiple parts of an application. The inclusion of meaningful, AI-assisted comments improves code readability and helps other developers quickly understand the purpose and behavior of the function. This approach supports cleaner software design practices, reduces code duplication, and makes future modifications simpler, highlighting the effectiveness of AI tools like GitHub Copilot in promoting modular and well-documented code.

Task 4: Comparative Analysis – Procedural vs Modular Approach (With vs Without Functions)

Prompt: #Comparison table or short analytical report with vs without functions

```
#Comparison Table
print("\nComparison Table:")
print("{<30} {<30}".format('Without Function', 'With Function'))
print("{<30} {<30}".format('Direct string slicing', 'Encapsulated in a function'))
print("{<30} {<30}".format('Less reusable', 'More reusable'))
print("{<30} {<30}".format('Less clear for multiple uses', 'Clearer for multiple uses'))
#Analytical Report
print("\nAnalytical Report:")
print("Using functions for string reversal enhances code reusability and clarity. While direct string slicing is str
```

Output:

```

Comparison Table:
Without Function          With Function
Direct string slicing     Encapsulated in a function
Less reusable            More reusable
Less clear for multiple uses  Clearer for multiple uses

Analytical Report:
Using functions for string reversal enhances code reusability and clarity. While direct string slicing is straightforward,
function-based approaches are more suitable for larger applications.
PS C:\Users\v1\OneDrive\Desktop\AI Coding-2026>

```

Observation: Comparison table or short analytical report with vs without functions

The comparison shows that code without functions is quick and simple for small tasks but becomes hard to manage as programs grow. Using functions makes the code more organized, readable, and reusable. It also makes debugging and maintenance easier. Overall, modular code is better suited for larger applications where clarity and maintainability matter

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different

Algorithmic Approaches to String Reversal)

Prompt: Different Algorithmic Approaches to String Reversal

```

02 C:\Users\v1\OneDrive\Desktop\AI Coding-2026\ass-1 (day-01).py
def reverse_string_iterative(s):
    """Reverses a string using an iterative approach."""
    reversed_str = ""
    for char in s:
        reversed_str = char + reversed_str
    return reversed_str
def reverse_string_recursive(s):
    """Reverses a string using a recursive approach."""
    if len(s) == 0:
        return s
    else:
        return s[-1] + reverse_string_recursive(s[:-1])
def reverse_string_slicing(s):
    """Reverses a string using slicing."""
    return s[::-1]
def reverse_string_builtin(s):
    """Reverses a string using built-in function."""
    return "".join(reversed(s))
# Example usage
input_string = "Hello, World!"
print("Iterative:", reverse_string_iterative(input_string))
print("Recursive:", reverse_string_recursive(input_string))
print("Slicing:", reverse_string_slicing(input_string))
print("Built-in:", reverse_string_builtin(input_string))

```

OUTPUT:

```

Analytical Report:
Using functions for string reversal enhances code reusability and clarity. While direct string slicing is straightforward,
function-based approaches are more suitable for larger applications.
Iterative: !dlroW ,olleH
Recursive: !dlroW ,olleH
Slicing: !dlroW ,olleH
Built-in: !dlroW ,olleH

```

Observation: Different Algorithmic Approaches to String Reversal.

The two string-reversal methods work differently and have different performance levels. The loop-based method reverses the string step by step, which makes it easy for beginners to understand, but repeated string concatenation slows it down for large inputs. The slicing method, on the other hand, reverses the string in one simple operation, making it faster, cleaner, and more efficient. While the loop approach is good for learning basics, slicing is better suited for real-world and performance-focused code.

